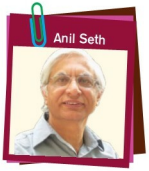

Exploring Big Data on a Desktop: Archiving and Retrieving Emails using HBase and Phoenix

In this column, the author discusses how large volumes of mail can be archived and retrieved using Big Data tools, HBase and Phoenix.

One pressing challenge for any organisation is the storage and retrieval of emails. There are some obvious fields, like the subject, the sender and the content, which need to be searched. However, the header lines may be very important as well. For example, it is easy to fake the sender, so the server that sent the mail may be significant.

An email's core components are the header, the content and the attachments. Each line of the header is a key-value pair, with each key representing a header field. There are some fields that are visible to the user, like the subject, the sender, the recipients, the date and time. Other fields may be examined in case of problems, and a user usually needs to make an additional effort to look at them on most graphical mail clients.

Planning for storage

Typically, each email is stored as a record in a data store. The list of possible header fields is very large. Mapping each one to a column would just not be convenient.

This is where the concept of the column family in HBase can provide an answer. For example, you may create an HBase table with the following column families:

- Envelope (header fields like sender, recipients, subject, date)
- Header (header fields not included in the envelope)
- Content
- Attachment

Create the HBase table from the HBase shell as follows:

```
[fedora@h-mstr hbase-0.96.2-hadoop2]$ bin/hbase shell
hbase(main):001:0> create 'emails','envelope','header','co
ntent','attachment'
0 row(s) in 5.0390 seconds
=> Hbase::Table - emails
hbase(main):002:0> exit
```

Loading emails

Python has a very versatile mailbox module for working with various mail formats on the disk. As an illustration, consider messages stored in the mbox format. A message may have multiple parts and each part may be another message or an attachment. In order to keep the code as simple as possible, convert and store each part as a string. Ideally, the unique key for each email would not be a UUID as has been used in this example. It would start with a value that is useful for limiting the search of emails. For example, in a corporate set-up, it may be prefixed with the department and date. As in the previous article in this series, use *happybase* to connect to the thrift server of HBase. The following code in *'load_mbox_file.py'* illustrates the idea.

```
#! /usr/bin/python
import mailbox
import happybase
import uuid
import sys
ENVELOPE=['To','From','Subject','Return-Path','Date']
# A simple approach to mutipart
# Store each part as a string ignoring the type of content
def multipart_payloads(columns, msg):
    num = 1
    for part in msg.get_payload():
        columns['content:' + str(num)] = part.as_string()
        num += 1

def store(table,msg):
    # create a unique id for the row
    row_id = str(uuid.uuid1())
    # Process the headers
    columns = {}
    # some header keywords may appear multiple times
    for key in set(msg.keys()):
        if key in ENVELOPE:
            cf = 'envelope:' + key
        else:
            cf = 'header:' + key
        # Get all the entries for a key as a list
        # Store as a string.
        # Can use eval to get the list back.
        columns[cf] = str(msg.get_all(key))
    # Multipart?
```